

Inventory & Order Management System

Aplikasi web multi-gudang dengan 3 peran, disusun berlapis (Controller → Service → Repository), dengan integritas stok yang terjaga saat proses berjalan bersamaan.

DISUSUN OLEH

Arief Aryudi Syidik

PROGRAM

PT Neuronworks Indonesia

EDISI

Participant Guide 1.0

SESI

04 September 2026

01 Agenda Sesi

12–15 MENIT

Demo Produk

Docker start, login 3 role, alur PO→SO→ledger, validasi/failure, search/pagination, dashboard, responsive.

5–7 MENIT

Engineering Evidence

Class diagram, ADR, unit + integration test, static analysis, refactoring log.

10–12 MENIT

Diskusi & Masukan

Critique exercise, penjelasan ARCH-02, penelusuran diagram→kode, dan sesi tanya jawab.

Masalah yang Ingin Diselesaikan

Tim gudang dan sales membutuhkan aplikasi untuk mencatat produk, mengelola stok di beberapa gudang, memproses pembelian dari supplier serta penjualan ke customer — dan memastikan angka stok tetap bisa dipertanggungjawabkan, termasuk saat dua proses berjalan bersamaan.

TUJUAN

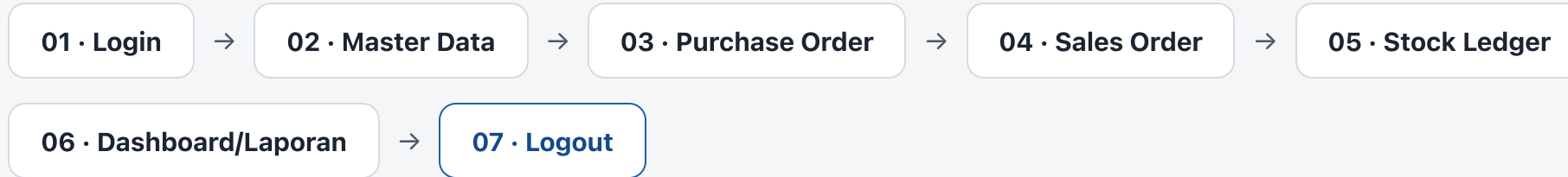
- ✓ Merancang arsitektur berlapis yang dapat diuji
- ✓ Menjaga integritas data pada operasi konkuren
- ✓ Menerapkan Clean Code & Clean Architecture secara nyata
- ✓ Mengomunikasikan keputusan desain dengan jelas

03 Tiga Peran, Tanggung Jawab Terpisah

AKTIVITAS	ADMIN	SALES	WAREHOUSE STAFF
Membuat & mengajukan Sales Order	Boleh	Boleh, milik sendiri	Tidak
Menyetujui / menolak Sales Order	Boleh	Tidak — termasuk order sendiri	Tidak
Goods issue (pengeluaran stok)	Boleh	Tidak	Boleh
Goods receipt (penerimaan stok)	Boleh	Tidak	Boleh

Aturan ditegakkan di **authorization layer server**, bukan hanya disembunyikan lewat UI — pembuat order tidak pernah bisa menjadi penyetujunya sendiri.

04 Login sampai Logout



Setiap pergerakan stok (goods receipt & goods issue) ditulis ke `stock_ledger` — dashboard dan laporan dihitung dari data ini, bukan angka statis.

05 Tumpukan Teknologi

AREA	DIPAKAI
Frontend	HTML semantik · CSS buatan sendiri · Vanilla JS + Fetch API
Backend	PHP 8.2+ Native OOP · DIP pada boundary repository
Database	MySQL 8 · PDO prepared statement · transaksi eksplisit
Environment	Dockerfile · Docker Compose · .env.example
Testing	PHPUnit (Unit + Integration) · PHPStan · PHP_CodeSniffer

Tanpa framework backend

Tanpa ORM

Tanpa framework JS

Controller

HTTP / routing

app/Controller

Service

Business rule

app/Service

Repository Interface

Boundary — Dependency Inversion

app/Repository/*Interface

MySQL / In-Memory

Akses data — implementasi ganda

app/Repository/MySQL*

Dependency mengarah ke bawah lewat interface; Service menerima Repository lewat constructor injection — tidak ada `new PDO()` tersembunyi di business logic, sehingga bisa diuji memakai implementasi in-memory.

07 Concurrency-Safe Stock

Dua goods issue untuk produk & gudang yang sama, diproses hampir bersamaan —

`SELECT ... FOR UPDATE` mengunci baris sampai transaksi pertama selesai.

SO-Fulfill A

lock → decrement → commit

SO-Fulfill B

menunggu row lock...

lanjut, cek ulang stok

Pilihan: pessimistic row lock InnoDB — bukan optimistic locking (butuh retry) atau mutex aplikasi (infrastruktur tambahan). Lihat [docs/architecture/adr-002-concurrency-safe-stock.md](#).

08 Hasil Test & Static Analysis

22

UNIT TEST · 39 ASSERTIONS

4

INTEGRATION TEST · 24 ASSERTIONS

0

PHPSTAN ERROR · LEVEL 5

0

PHPCS ERROR · PSR-12

3+

AREA LOGIC DIUJI TERISOLASI

MySQL

INTEGRATION TEST KE DB NYATA DI DOCKER

Diverifikasi ulang hari ini — hijau di kedua suite, dijalankan lewat `composer test` dan terhadap MySQL nyata di Docker.

09 Bukti Penalaran Desain

Class diagram	initial vs as-built
ADR	adr-001 Repository pattern · adr-002 Concurrency
Refactoring log	3 entri smell → teknik → before/after
Audit SRP	1 pelanggaran, cara perbaikan

CRITIQUE EXERCISE

Analisis God Method fiktif: pelanggaran SRP, DIP, OCP, SQL injection, hardcoded credential — dibahas tanpa implementasi, murni penalaran.

TECH-DEBT REGISTER

10 keterbatasan dicatat secara jujur — mis. `ApiController` masih memanggil Repository langsung, belum lewat Service.

10 Deployment & Data Seed

5

AKUN DEMO · 3 ROLE

32

PRODUK, VARIASI REORDER POINT

2

GUDANG

13

PURCHASE ORDER

12

SALES ORDER

**docker compose
up --build**

SATU PERINTAH, DARI KONDISI BERSIH

Endpoint JSON: `GET /api/products/{sku}/availability` → 200/401/404 sesuai kontrak.

11 Status Proyek

SYARAT KELULUSAN

≥ 80

dan tidak terdapat critical failure.

- ✓ Alur login–PO–SO–stok berfungsi penuh
- ✓ Goods issue/receipt transaksional (row lock InnoDB)
- ✓ Segregation of duties teruji di server
- ✓ Unit + integration test lulus, nol critical static error
- ✓ Class diagram sesuai kode aktual, ADR & refactor log lengkap

Terima Kasih atas Waktu dan Kesempatannya

Kami menyadari masih banyak ruang untuk perbaikan. Kritik, pertanyaan, dan masukan sangat kami harapkan untuk penyempurnaan proyek ini ke depan.

REPO
ioms/

DEMO
localhost:8080

AKUN
lihat README.md

KONTAK
Arief Aryudi Syidik